IT1244

Artificial Intelligence: Technology & Impact

Week 4 Tutorial 2

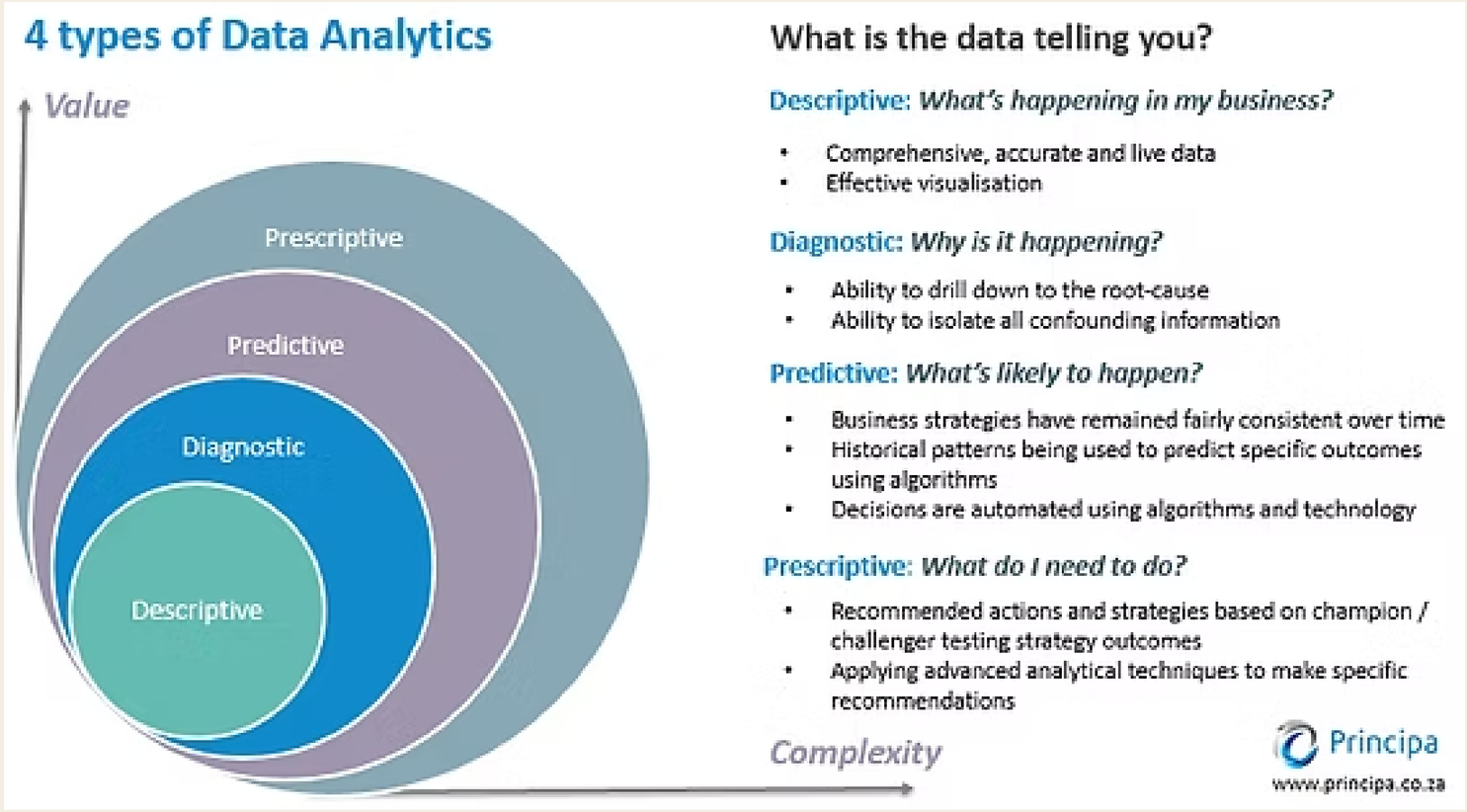
TODAY'S AGENDA

1. Recap of concepts
2. Pollev Questions
3. Discussion of tutorial questions

RECAP

THREE LEVELS OF ANALYTICS

1. **Descriptive Analytics**
   - What happened?
   - Clustering, PCA
   - Data visualization
2. **Predictive Analytics**
   - What will happen?
   - Classification, regression
   - Forecasting
3. **Prescriptive Analytics**
   - What should we do?
   - Search, planning, optimization
   - Decision support



SEARCH ALGORITHMS IN AI

What is a search algorithm?

- a systematic process used by an AI agent to navigate through a "problem space" to find a solution or reach a goal state

Input

- Problem Formulation: States, Actions, Transition model, Goal test, Path cost, etc

Output

- Sequence of actions to reach the goal state from the current state

SEARCH ALGORITHMS IN AI

Problems that use search algorithm:

Finding ‘best’ path for you

Amazon robotics to navigate warehouse

How to Make a Pathfinding Enemy NPC in Roblox Studio!

SEARCHES

Formulating a search problem

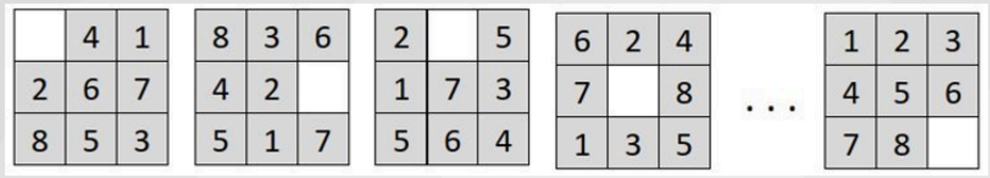
Component	Description
State Space	Set of all possible configurations a system can be in at any point of time.
Action Space	Set of all possible moves **given the current state**.
Successor Function	Given current state and a possible action, return the next state / successor.
Start State	The state from which the search algorithm starts.
Goal Test	The condition that determines whether the given state is a goal state.
Path Cost	A numerical cost associated with a given path.

SEARCHES

Formulating a search problem: 8 Puzzle Game

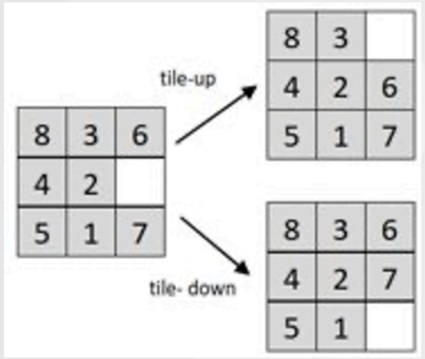
Input:

- State space:



- Action space: { tile-up, tile-down, tile-right, tile-left}

- Successor function:



- Start state:

6	2	4
7		8
1	3	5

- Goal test:

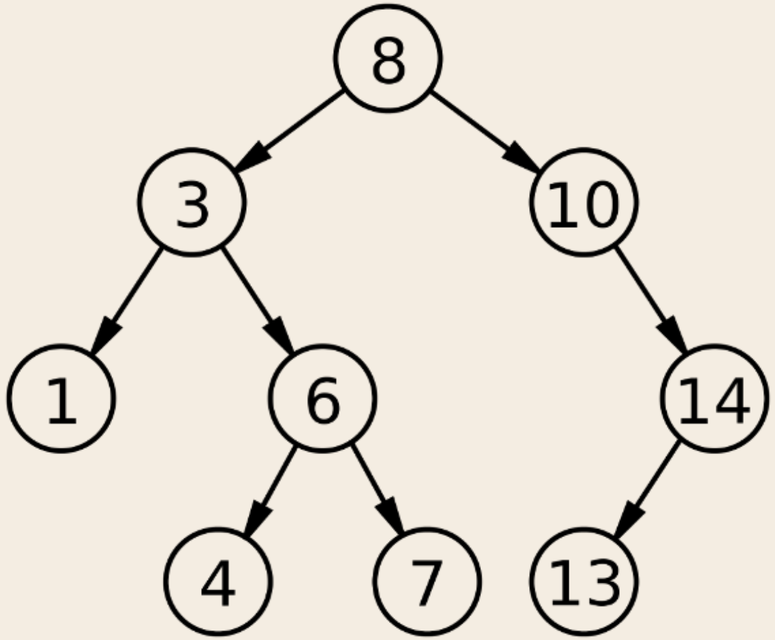
1	2	3
4	5	6
7	8	

- Path cost : number of steps in path (step cost of 1)

SEARCHES

Search Tree

- Root node = start state
- Child nodes = successor states
- Edges = actions from one state to another and cost of that action

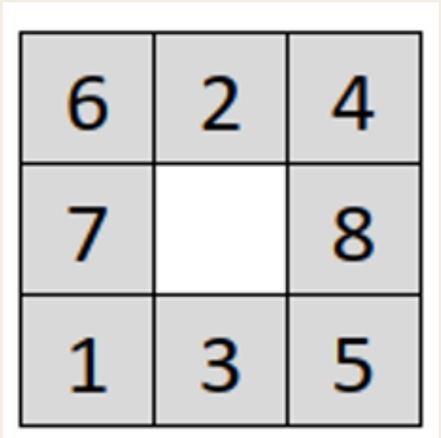


https://tristanpenman.com/demos/n-puzzle/

SEARCHES

State vs Node

- **State**: Specific configuration / Snapshot of the problem
- **Node:** Data structure that the state is stored in computer memory. Contains its parent, child, depth, path cost etc information about that state



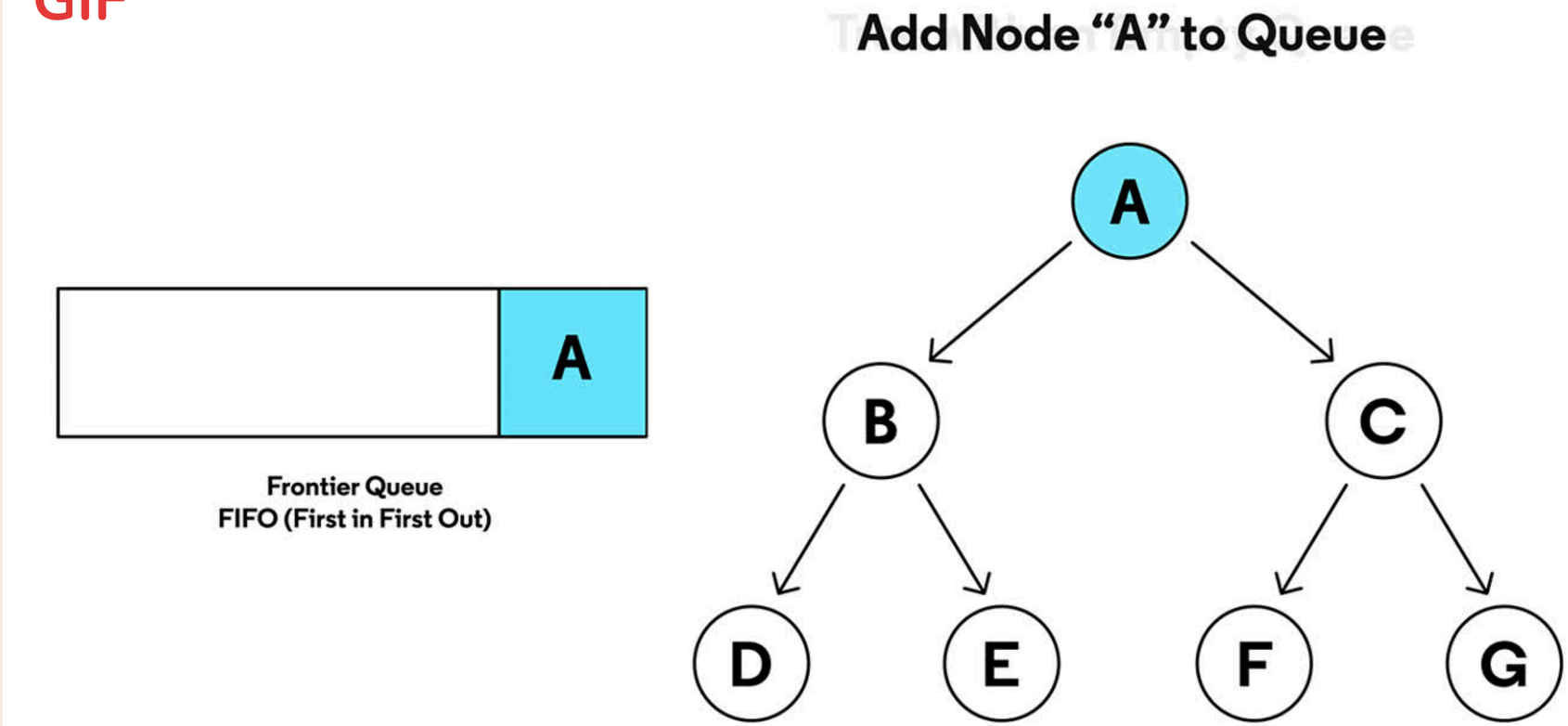
State

Node

SEARCHES

Frontier: Store nodes that have been discovered but not yet expanded.

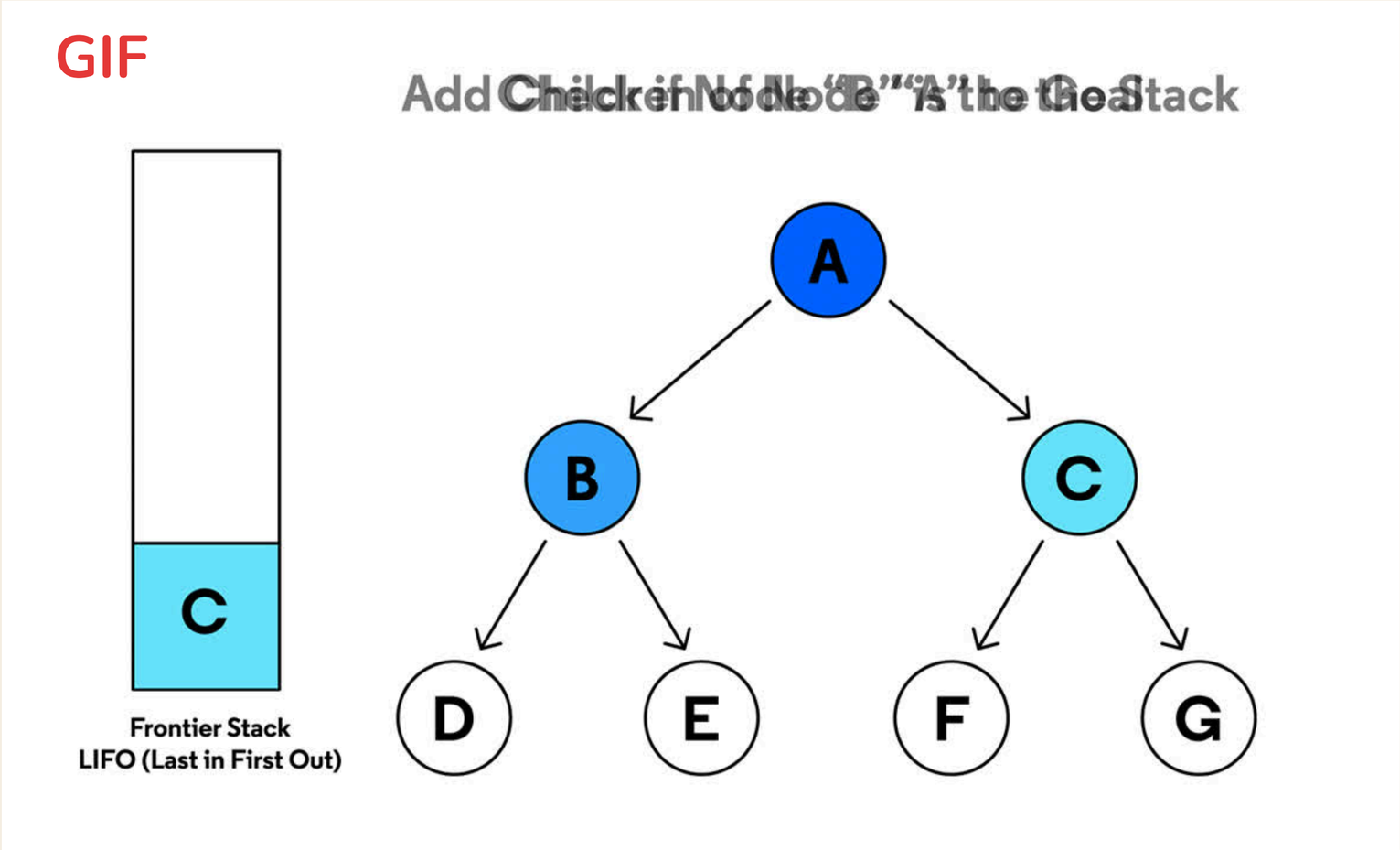
GIF



Breadth-first-search (BFS) - FIFO / Queue

SEARCHES

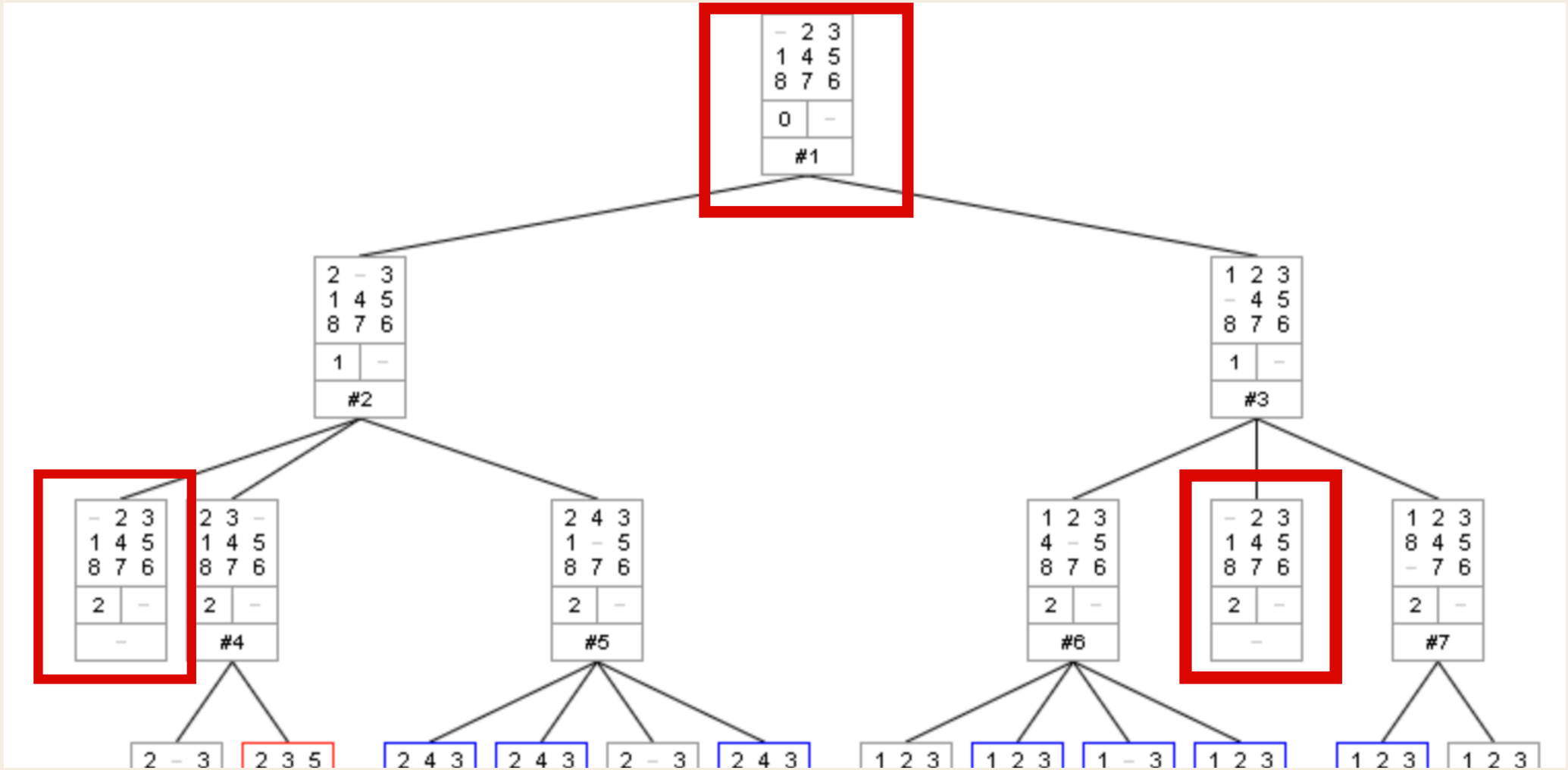
Frontier: Store nodes that have been discovered but not yet expanded.



Depth-first-search (DFS) - LIFO / Stack

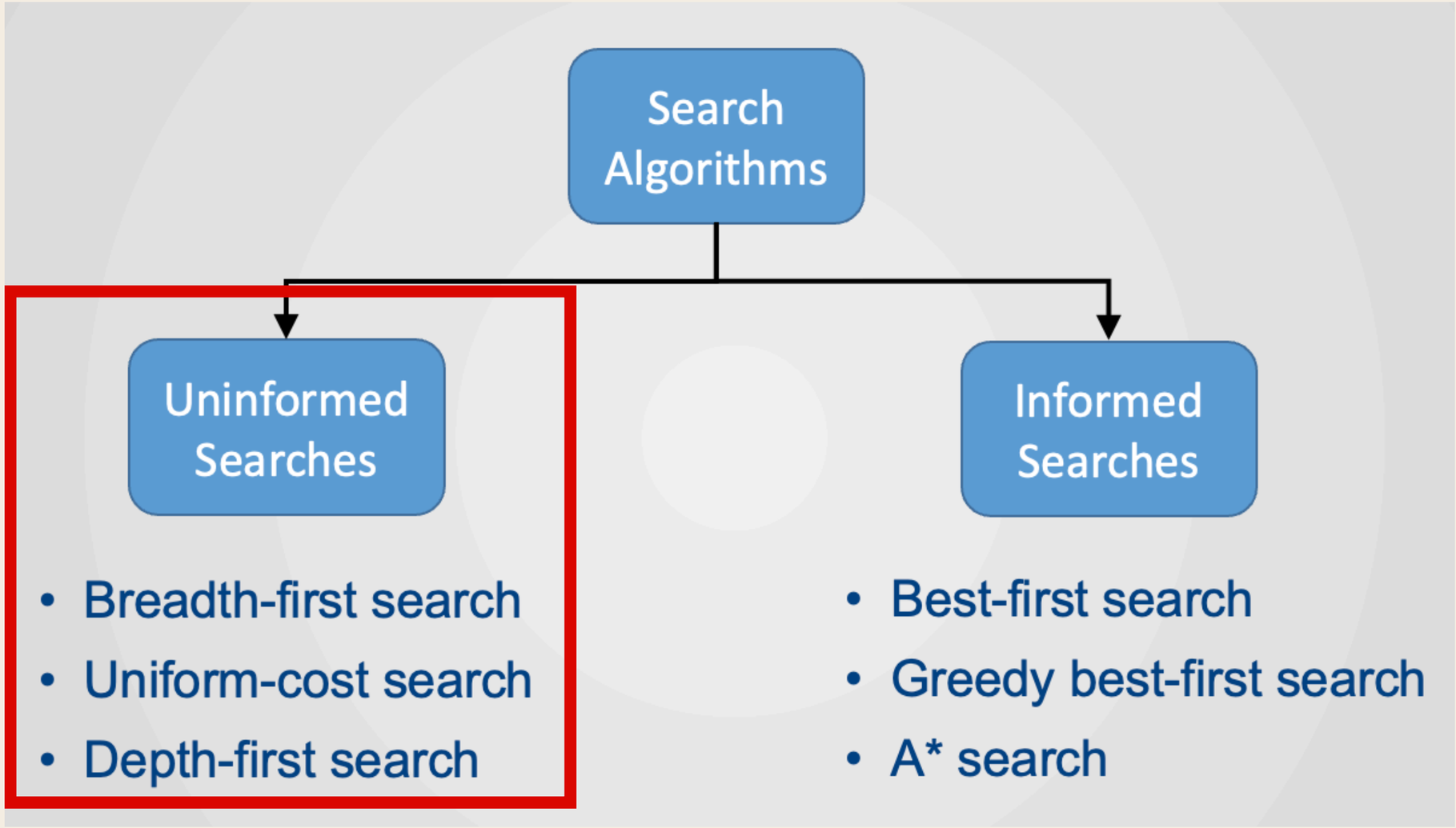
SEARCHES

Explored Set: set that stores already expanded states (so we don't revisit them)



SEARCHES

Types of search techniques:



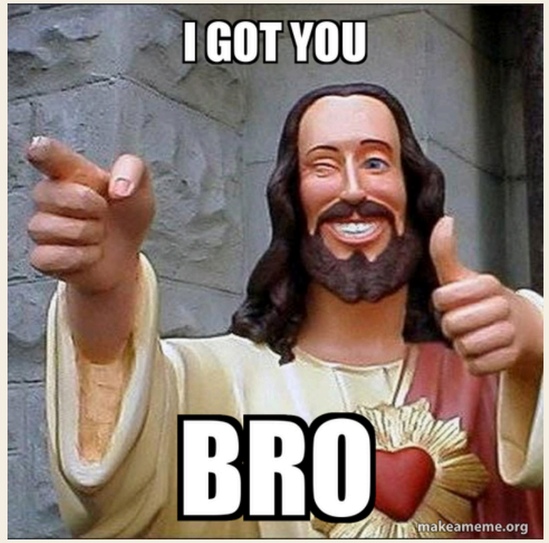
SEARCHES

Types of search techniques

Different methods to determine the order of picking node for expansion

Uninformed Search

- Blind search
- “I don’t know which node is closer to the goal bro”
- idk which station is closer → blindly explore all paths until I find one



Informed Search

- Guided by heuristics
- “It's ok bro, heuristic gave me an **estimate** of how close I am to the goal”
- When you do have information that can help estimate which is expansion **MAY** be better → use informed search

SEARCHES

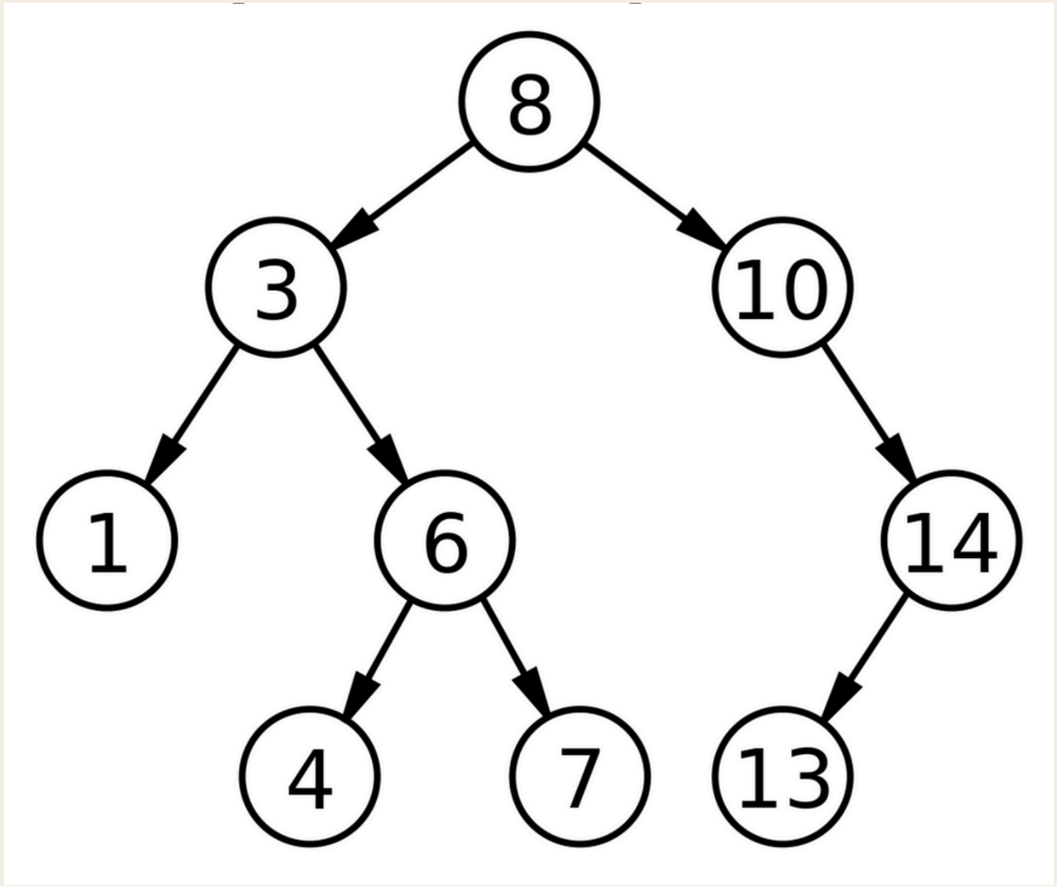
Evaluation of search algorithms

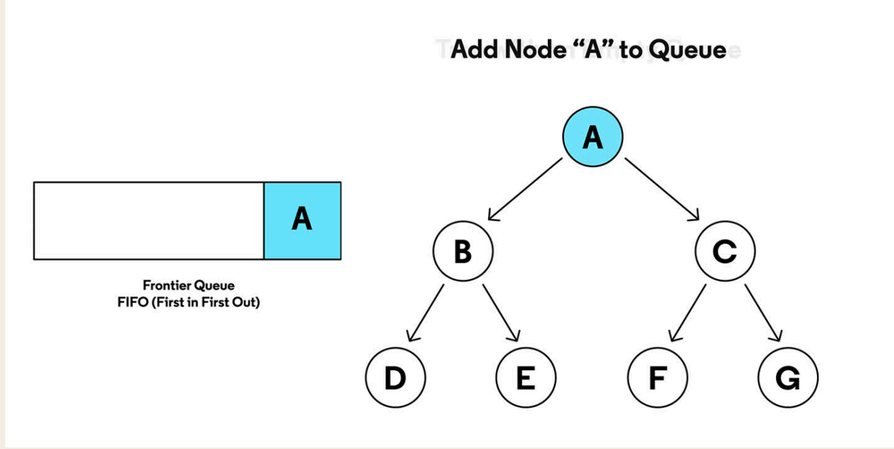
1. Completeness: **Guarantee of finding a solution** if there exists one for a given problem.
2. Optimality: **Guarantee of finding the best solution** (e.g. lowest path cost, shortest path, etc.)
3. Time Complexity: Time spent to find a solution.
4. Space Complexity: Memory space needed to perform the search.

Time and Space Complexity is not a big concern for this module! 🥳

SEARCHES

Breadth-First Search (BFS)





Characteristics

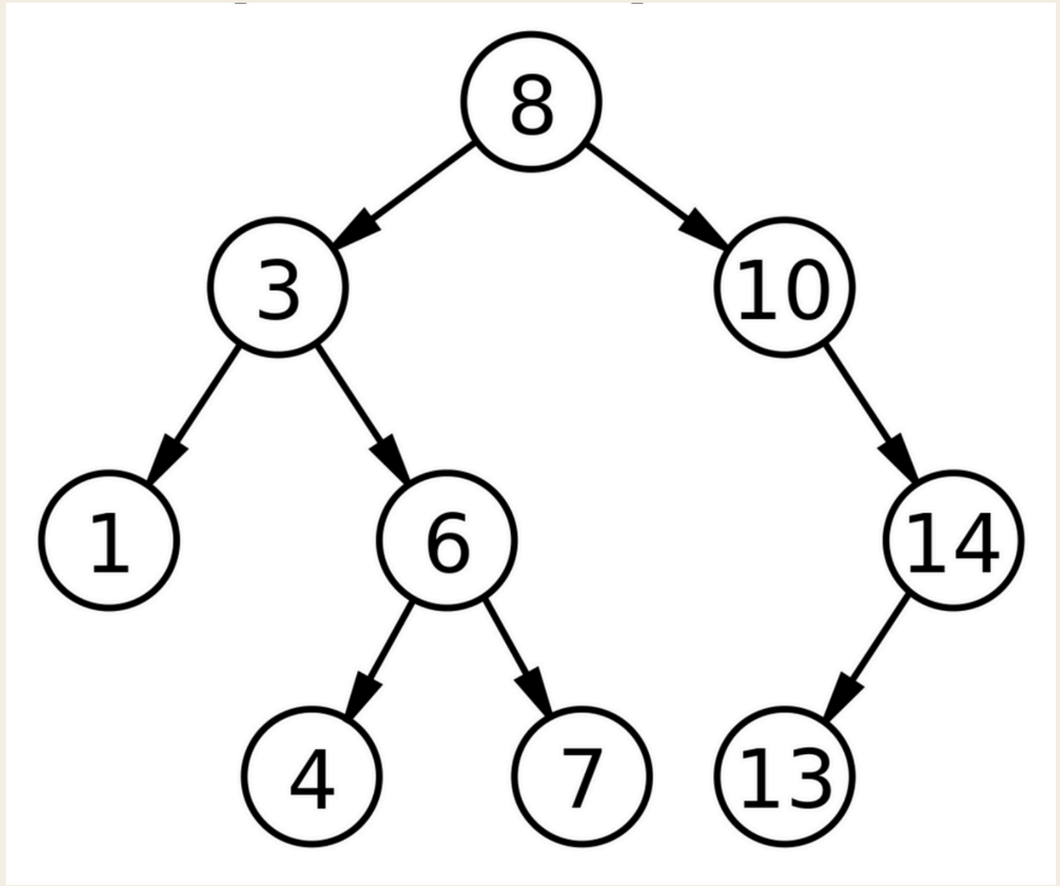
- Going level by level
- Frontier: Queue (FIFO)
- Result: {8, 3, 10, 1, 6, 14, 4, 7, 13}

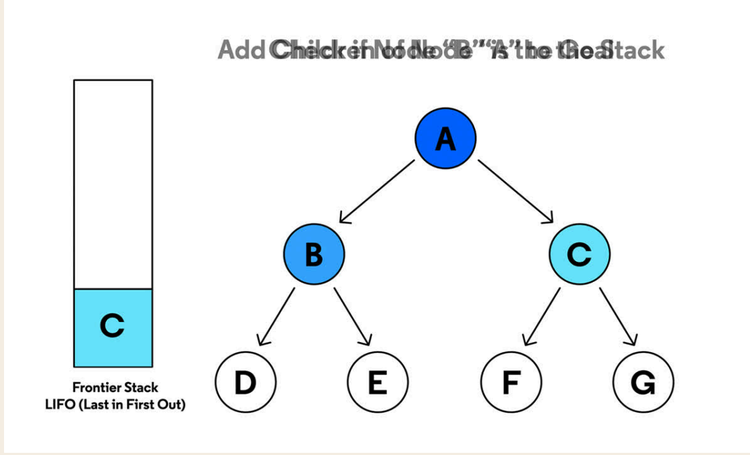
Properties

- **Complete:** Yes (if branching factor is finite)
- **Optimal:** Yes (if all step costs are equal)
- **Time & Space Complexity:** Takes up more space compared DFS (stores all nodes at the current level)

SEARCHES

Depth-First Search (DFS)





Characteristics

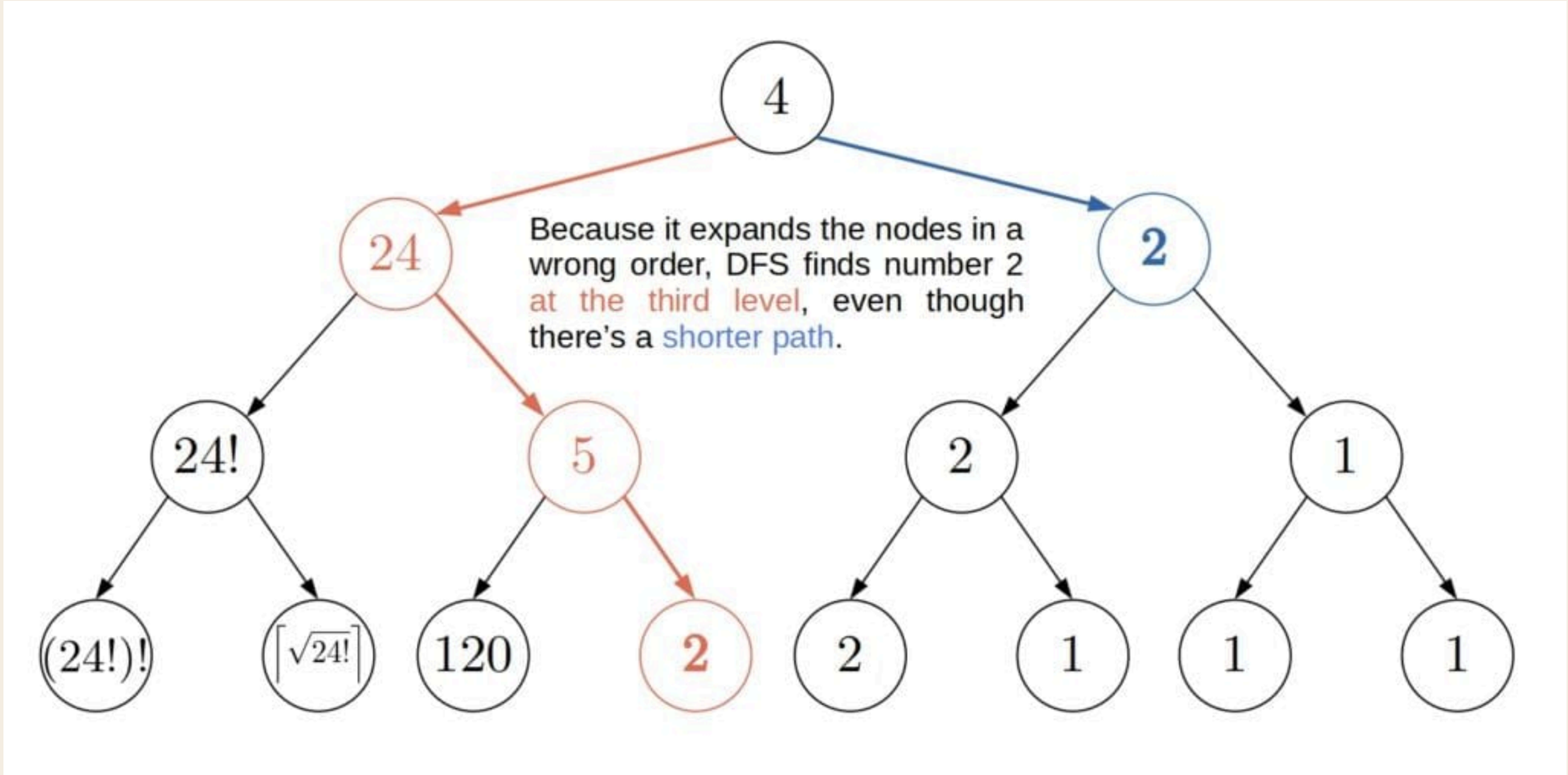
- Left until hits end → Backtrack
- Frontier: Stack (LIFO)
- Result: {8, 3, 1, 6, 4, 7, 10, 14, 13}

Properties

- **Complete:** Yes (if state space is finite)
- **Optimal:** **No** (there might be better solution(s) at shallower level)
- **Time & Space Complexity:** More memory-efficient for deep graphs because it only stores the nodes along the single path it is currently exploring

SEARCHES

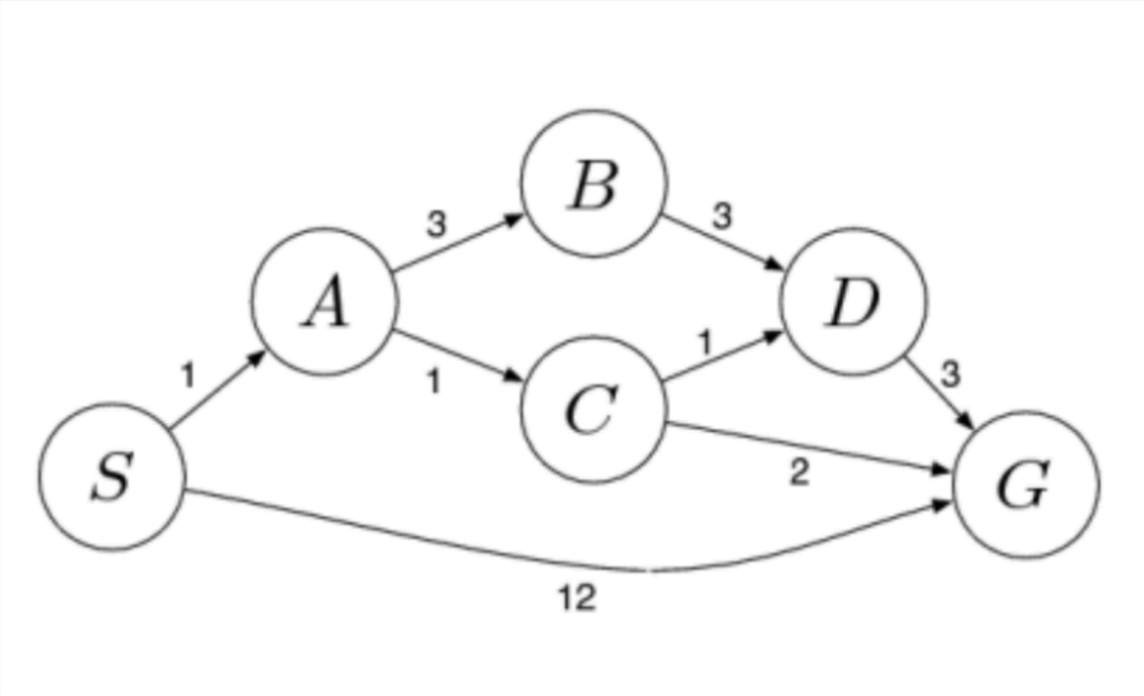
If you're confused on why it's optimal for BFS and not DFS



This is a better example to showcase why.

SEARCHES

Uniform Cost Search (UCS)



Characteristics

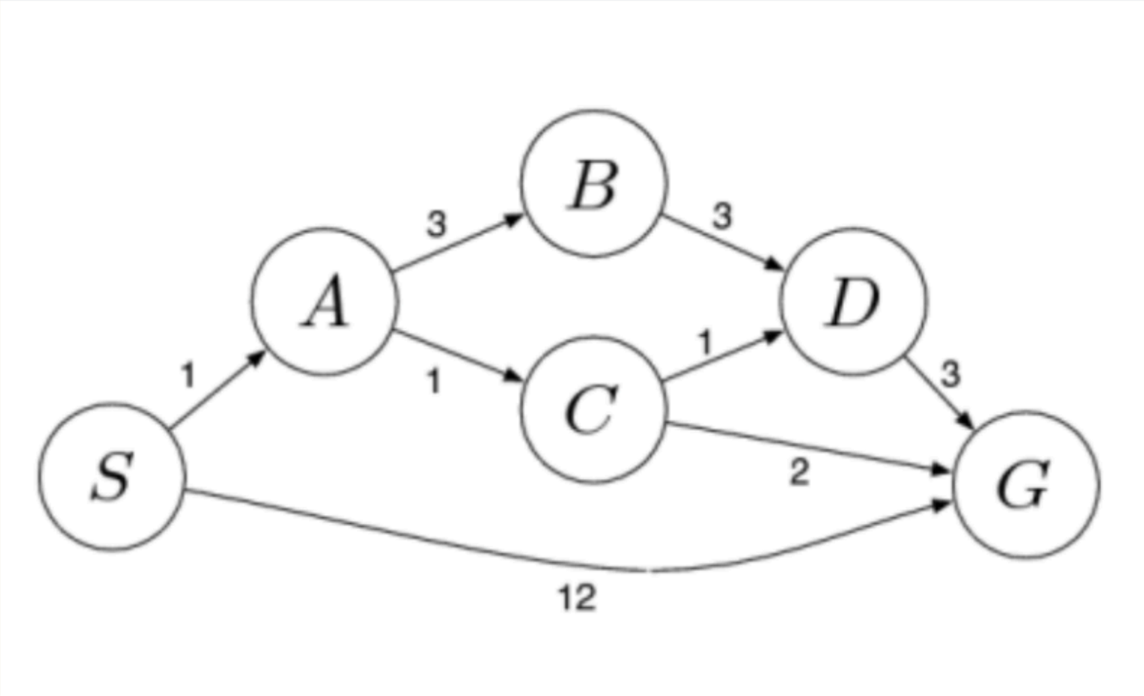
- Return most optimal ('cheapest cost') path
- Requires positive path cost
- Frontier: **Priority Queue** (elements arranged in decending priority)

Properties

- Complete: **Yes** (if branching factor is finite & all path costs are positive)
- Optimal: **Yes** (always; expands node at optimal path cost)
- Time & Space Complexity: **Worse than BFS**

SEARCHES

Uniform Cost Search (UCS)



1. Add S into Frontier

PQ = {(S, 0)}

2. Expand S

PQ = {(A, 1), (G, 12)}

3. Expand A

PQ = {(C, 2), (B, 4), (G, 12)}

4. Expand C (*update frontier if better path is found)

PQ = {(D, 3), (B, 4), (G, 4)}

5. Expand D (*don't update worse paths, SACDG has path cost 6)

PQ = {(B, 4), (G, 4)}

6. Expand B (*don't revisit expanded nodes (node D))

PQ: {**(G, 4)**}

7. Expand G (*Optimal path found: SACG, cost=4)

PQ: {}

SEARCHES

Uniform Cost Search (UCS) - FAQ

You mentioned that Uninformed is ‘blindly’ finding a path. But UCS chooses the one with the cheapest cost, that’s not ‘blindly searching’. Why is it being classified as Uninformed search?

Anyone know why? 🤔

SEARCHES

Uniform Cost Search (UCS) - FAQ

You mentioned that Uninformed is ‘blindly’ finding a path. But UCS chooses the one with the cheapest cost, that’s not ‘blindly searching’. Why is it being classified as Uninformed search?

If you go back to Slide 15, you’ll notice that “I don’t know which node is closer to the goal bro”. UCS is uninformed because the ‘cheapest cost’ is simply cost from the starting node, not cost to the goal.

We still have no information on how close the node is to the goal, unlike informed search which we’ll see next week. Hence, uninformed.

POLLEV QUESTIONS

Join by Web:

PollEv.com/kaironglee

Join by QR:

TUTORIAL QUESTION 1

Explain the terms “Narrow AI” and “Artificial General Intelligence” with examples.

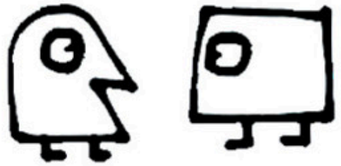
Anyone interested to share their findings? :D

TUTORIAL QUESTION 1

3 stages of AI

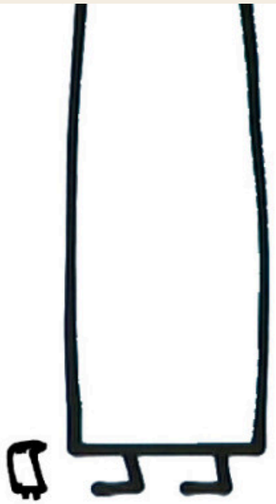
Narrow AI

Dedicated to assist with or take over specific tasks



General AI

Takes knowledge from one domain, transfers to other domain



Super AI

Machines that are an order of magnitude smarter than humans

Credit: Chris Noessel

TUTORIAL QUESTION 1

Narrow AI (Weak AI)

- AI designed for a **specific, limited task or range of tasks**, excelling within its specialized domain but unable to perform outside it
- Eg. Chess-playing AI, Facial Recognition

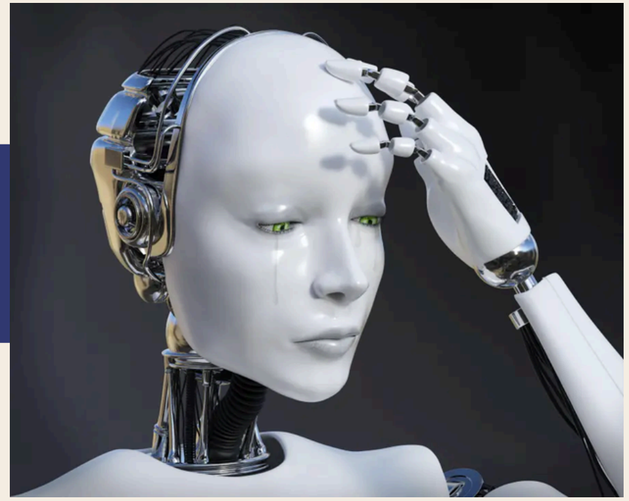
Artificial General Intelligence (Strong AI, General AI)

- AI that possesses the ability to **understand, learn, and apply knowledge across a wide variety of tasks** at a level equal to or exceeding human intelligence.

Q: Is ChatGPT an AGI?

TUTORIAL QUESTION 1

Q: Is ChatGPT an AGI?



As for now, not really.

- ChatGPT operates based on patterns and statistical probabilities in data, rather than having genuine contextual understanding
  - Hence it sometimes hallucinates
- ChatGPT cannot act completely independently to pursue goals over long periods (still needs prompting), which is a hallmark of AGI.

TUTORIAL QUESTION 2

Give an example task for (i) Descriptive analytics,

(ii) Predictive analytics and (iii) Prescriptive analytics.

Anyone interested to share? :D

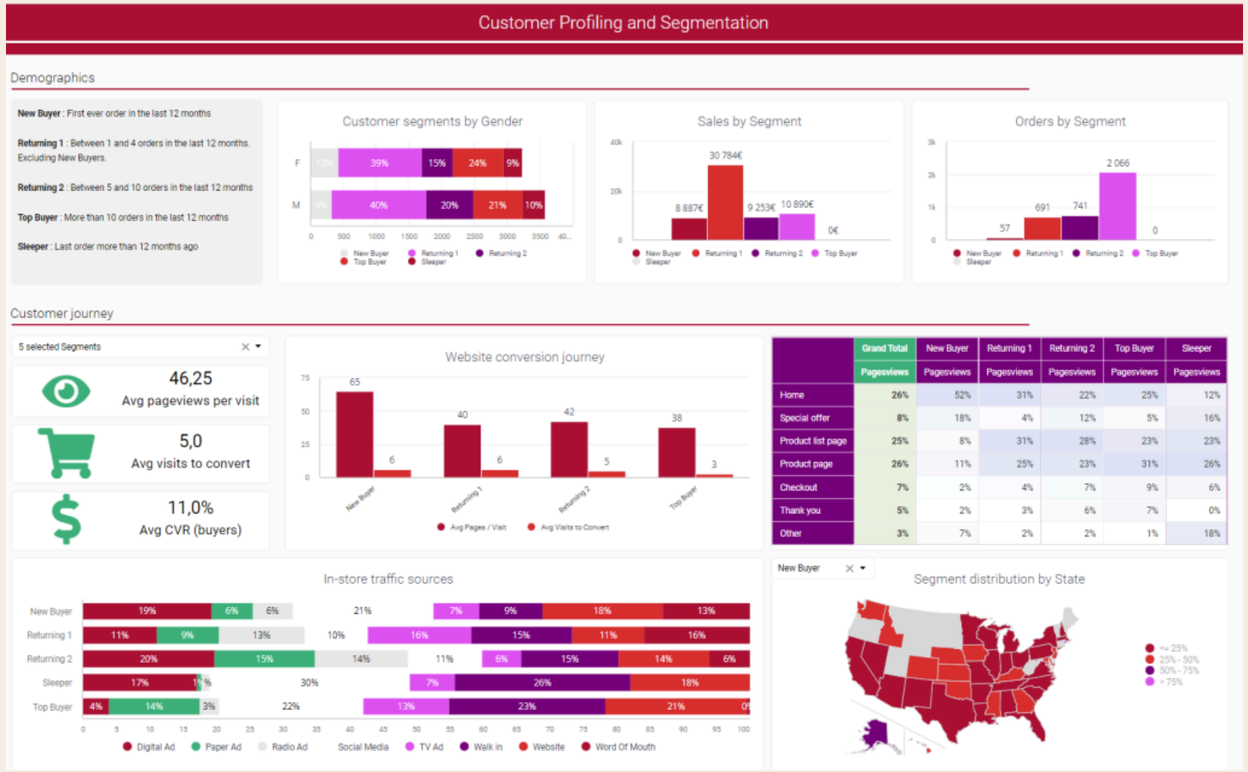
TUTORIAL QUESTION 2

Descriptive analytics (What happened?)

- Using visualization and aggregations to attempt to understand or make value of historical data

Examples

- Past Revenue and Sales Analysis
- Customer Behaviour Analysis
- Inventory Analysis



https://www.clicdata.com/blog/5-types-of-data-analytics-to-drive-your-business/

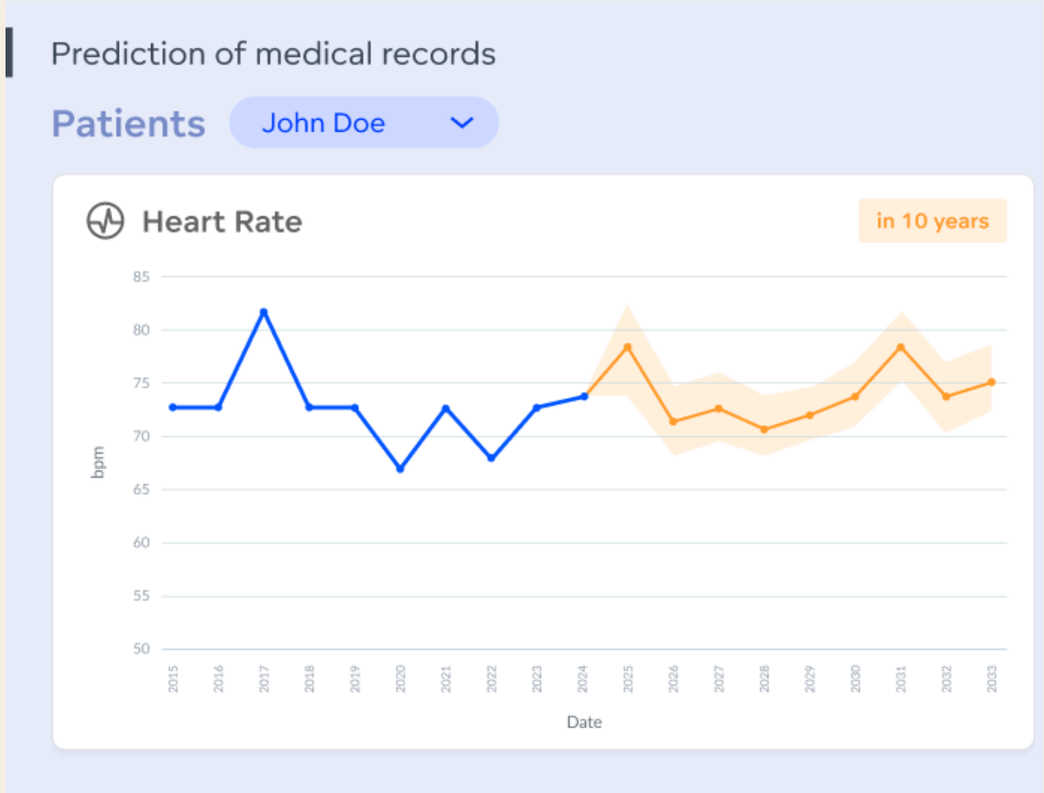
TUTORIAL QUESTION 2

Predictive analytics (What will happen?)

- Makes use of statistical algorithm, machine learning and other techniques to predict future events and outcomes

Examples

- Forecasting Future Sales to optimize inventory levels
- Identify at-risk patient to begin early treatment if at risk



https://www.clicdata.com/blog/5-types-of-data-analytics-to-drive-your-business/

TUTORIAL QUESTION 2

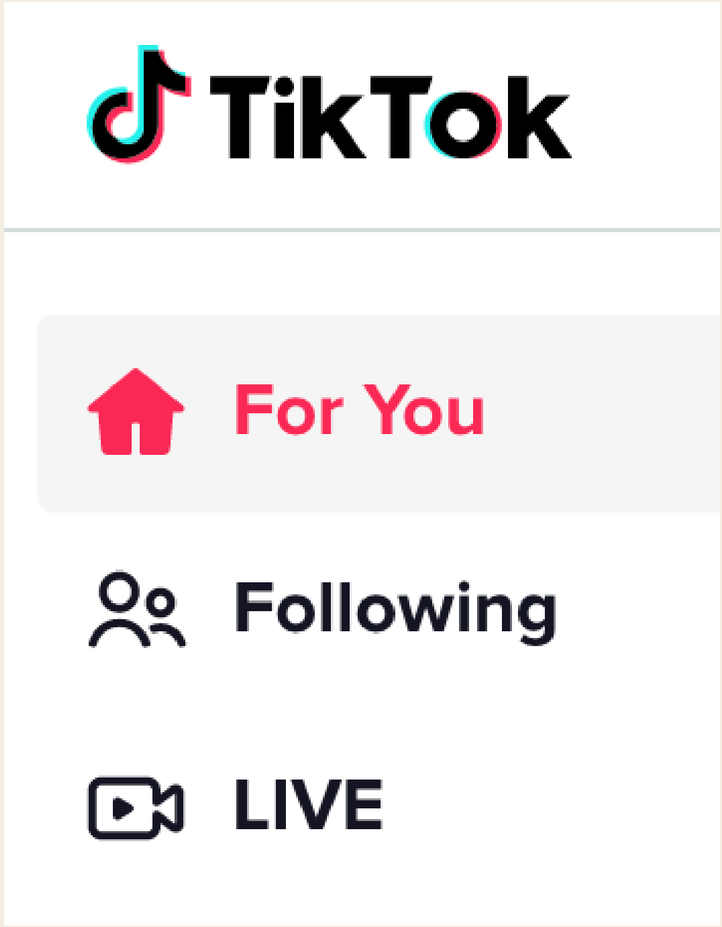
Prescriptive analytics (What should we do?)

- Makes use of descriptive and predictive analytics' results to recommend a course of action that helps achieve desired results

Examples

- Tiktok using past data of what you watched **(descriptive)** + predicts what you might watch based on similar genre **(predictive)**

→ recommends videos on your FYP (prescriptive)



https://www.clicdata.com/blog/5-types-of-data-analytics-to-drive-your-business/

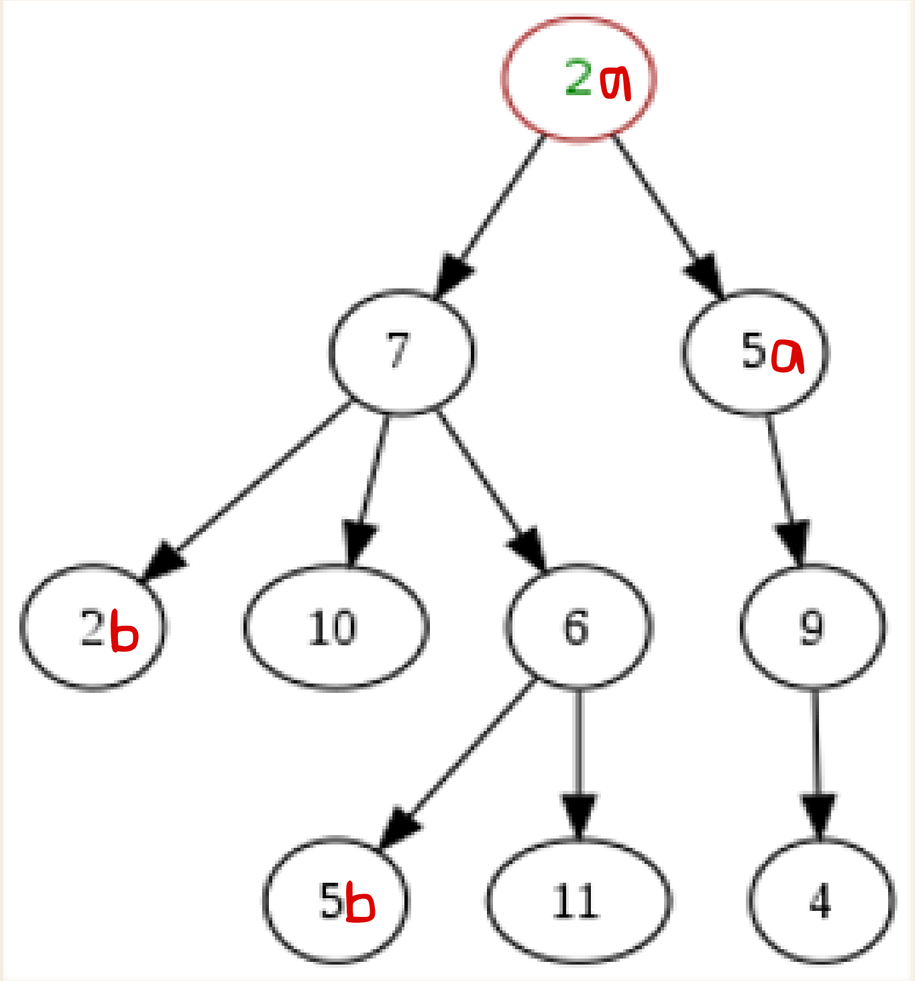
TUTORIAL QUESTION 3

Given a Tree data structure.

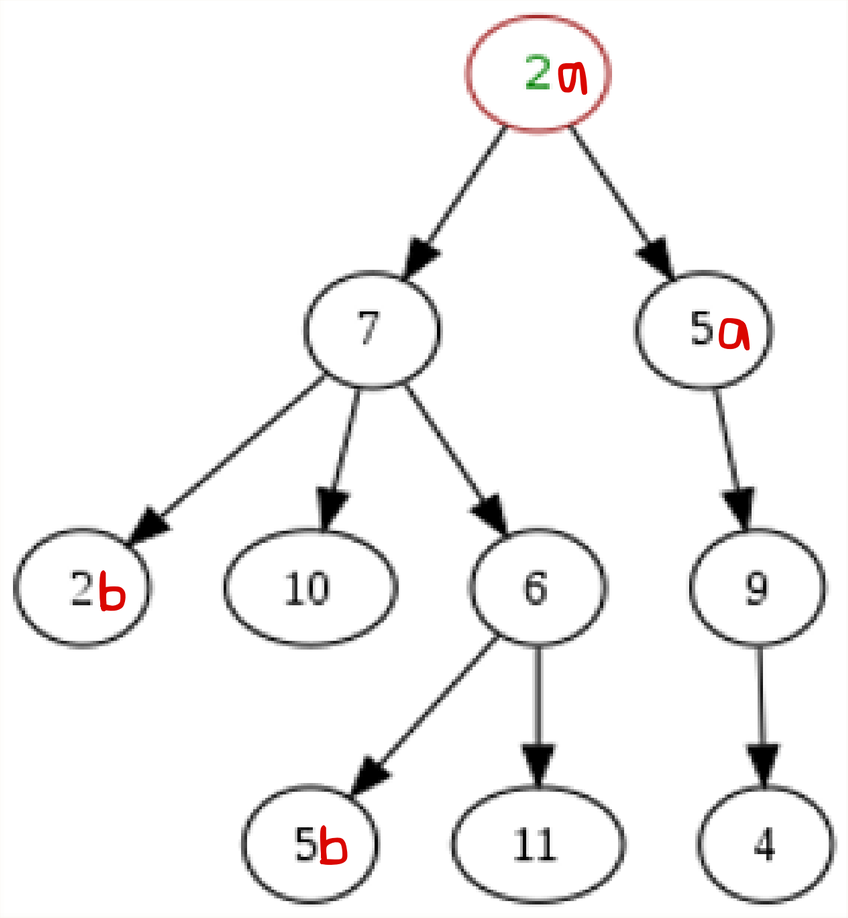
Root to node 4:

a.Show the nodes visited as output of **BFS**

b.Show the nodes visited as output of **DFS**



TUTORIAL QUESTION 3



BFS (Layer-by-layer)

2a → 7 → 5a → 2b → 10 → 6 → 9

→ 5b → 11 → 4

DFS (Left until cannot left anymore)

2a → 7 → 2b → 10 → 6 → 5b → 11 →

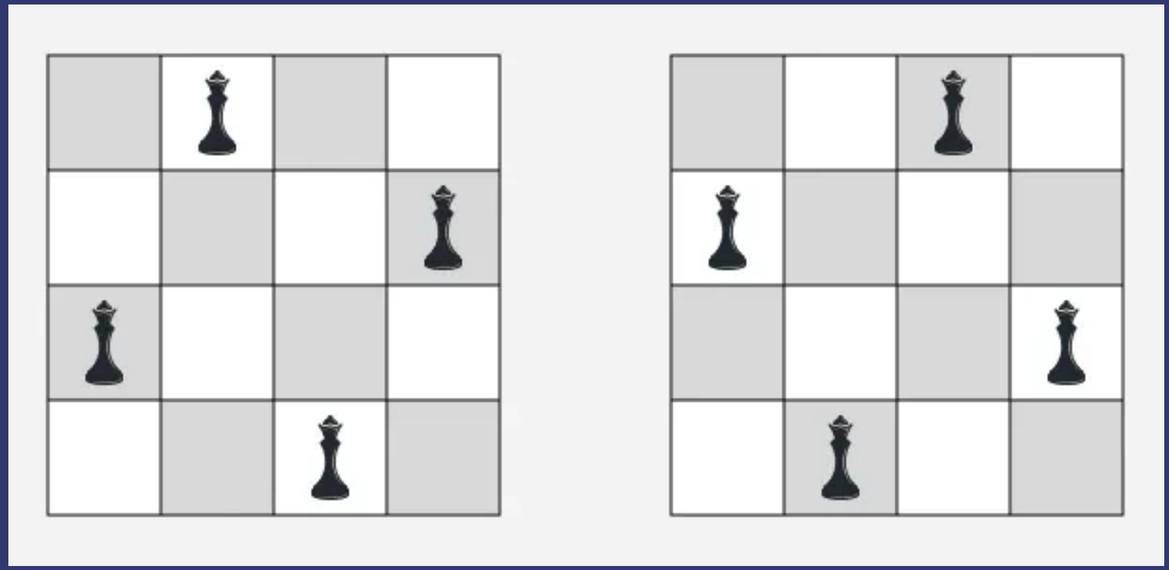
5a → 9 → 4

TUTORIAL QUESTION 4

Formulate a search problem for 4-Queen problem

The 4 Queens Problem: placing four queens on a 4 x 4 chessboard so that no two queens attack each other.

No two queens are allowed to be placed on the **same row, the same column or the same diagonal**.



TUTORIAL QUESTION 4

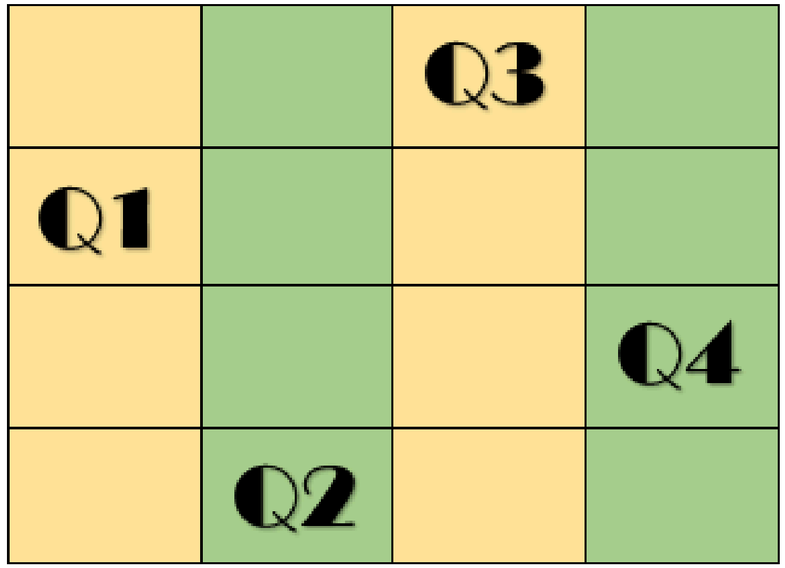
Fun Question before we start:

How many solutions does 4-queens have?

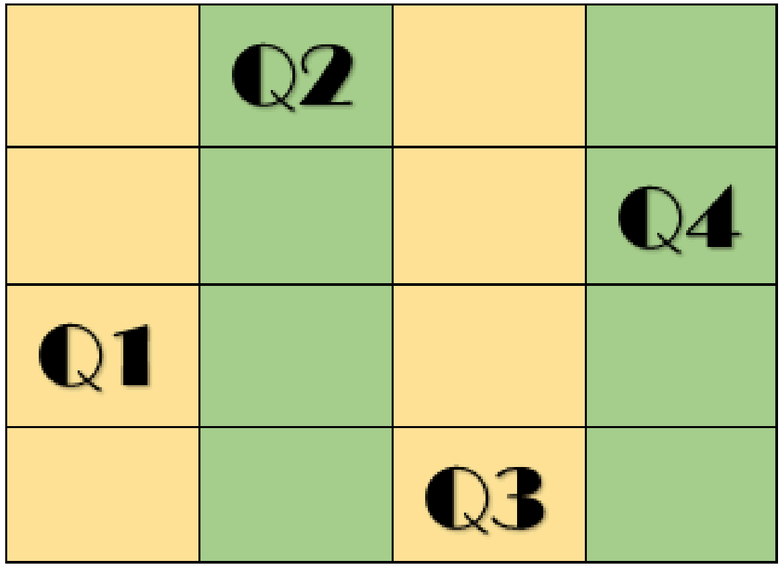
TUTORIAL QUESTION 4

How many solutions does 4-queens have?

Only 2 distinct answers!



Solution one



Solution two

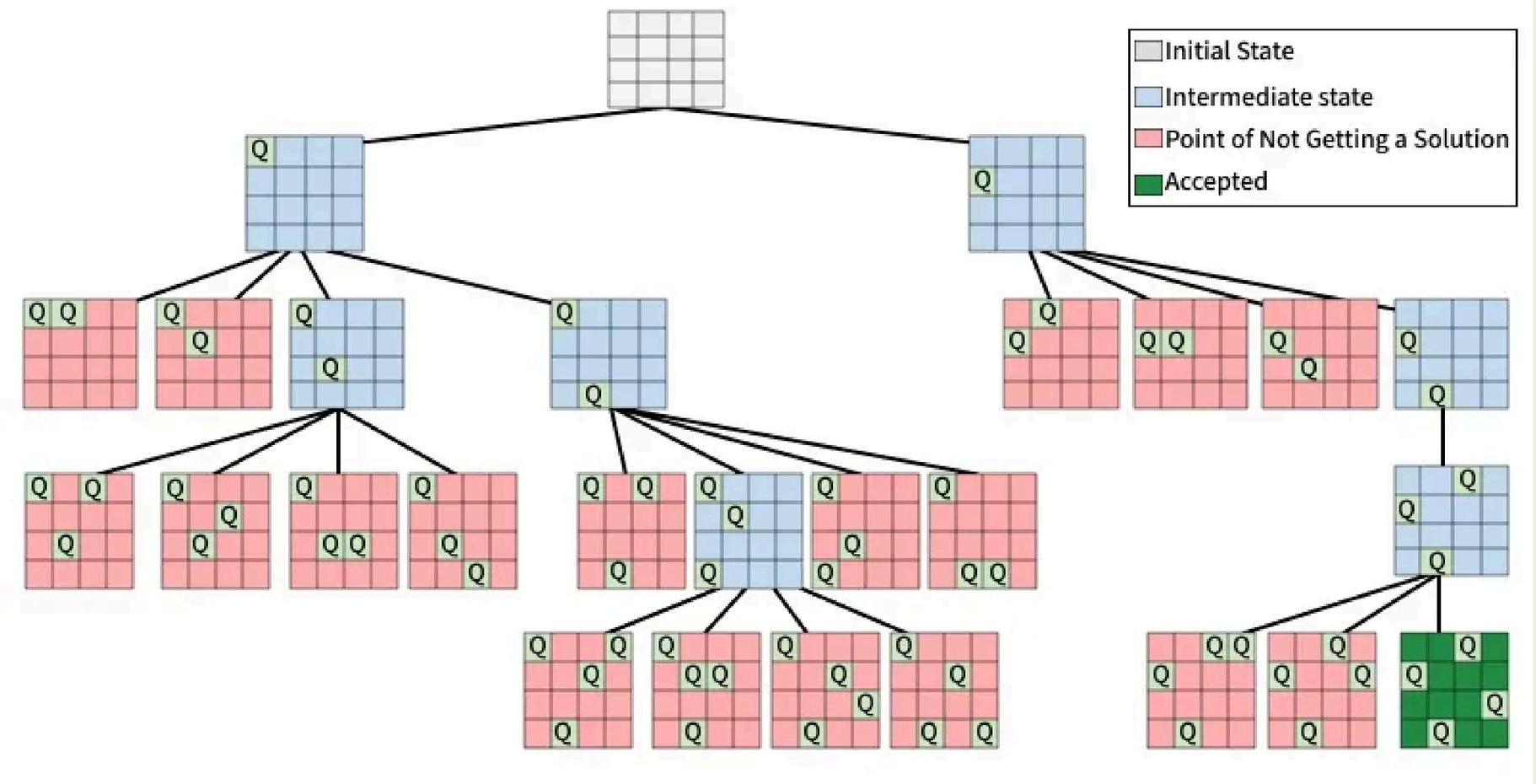
TUTORIAL QUESTION 4

Formulate a search problem for 4-Queen problem

Component	Description
State Space	{All configurations on 4x4 chessboard} *Having 0, 1, 2, 3, or 4 queens in the board can also be considered as partial state in this example.*
Action Space	{All the actions that we can take **given a current state of board**}
Successor Function	Function that takes current state + an action (placing a queen somewhere) and outputs the new board configuration.
Start State	Empty board
Goal State	All queens are placed with no any pair of queens attacking each other
Path Cost	0, Not Applicable (Week 3 Lecture Slide 17)

TUTORIAL QUESTION 4

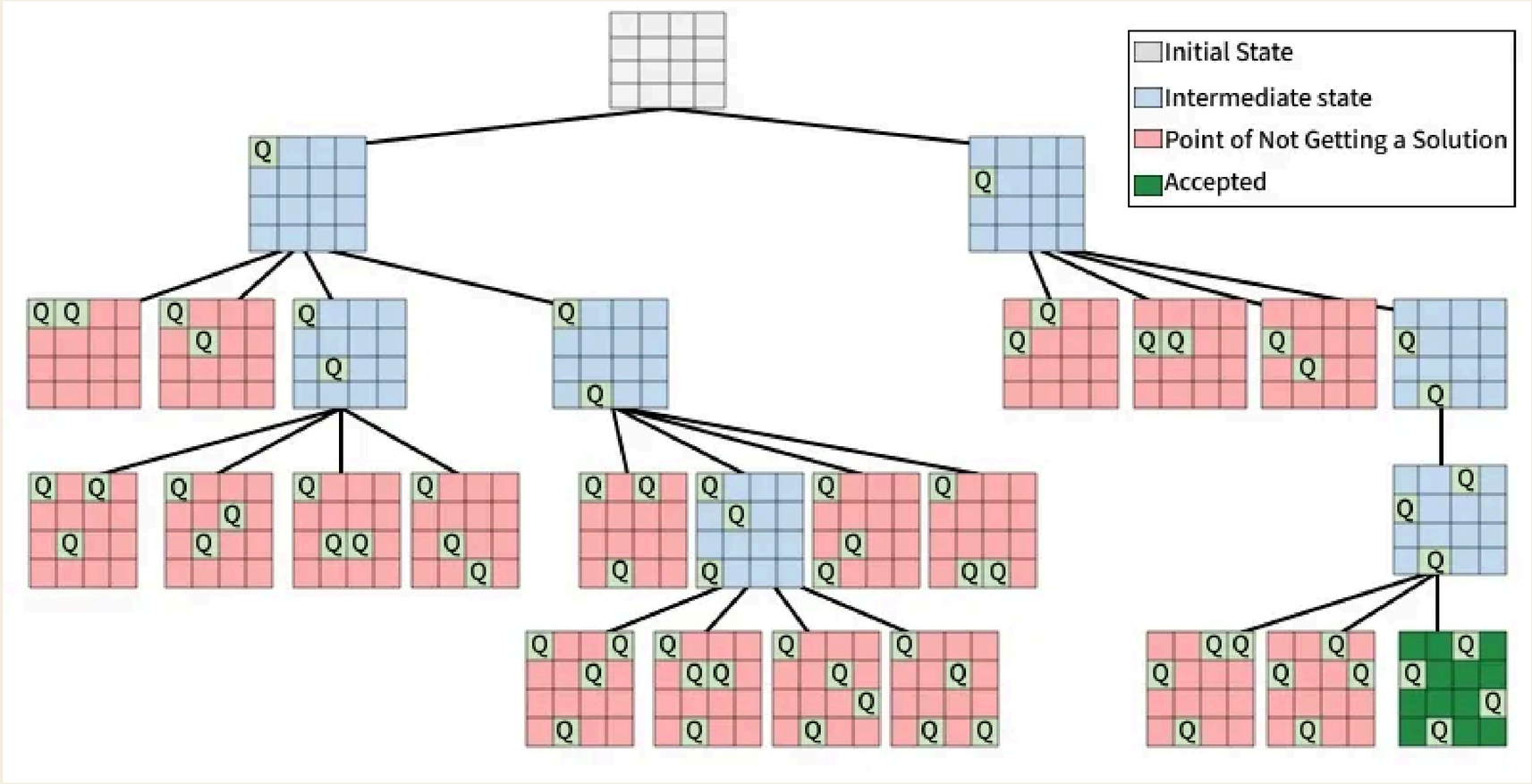
4-queens Search tree



Note: The above implementation follows the convention in Assignment 2, i.e., column by column

TUTORIAL QUESTION 4

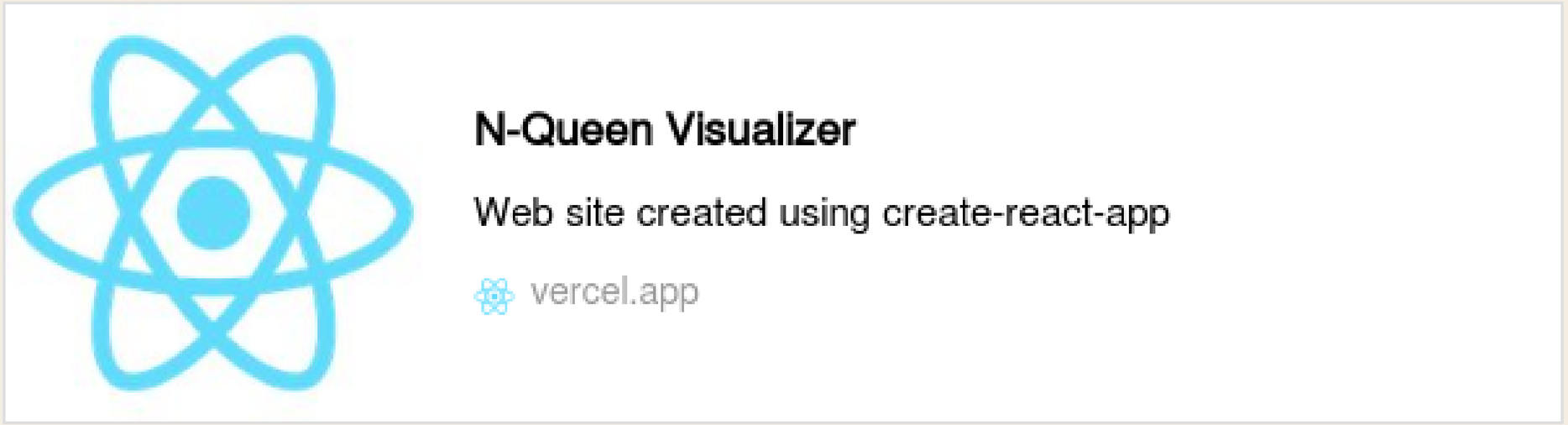
4-queens Search tree



Is this search tree a BFS or DFS?

TUTORIAL QUESTION 4

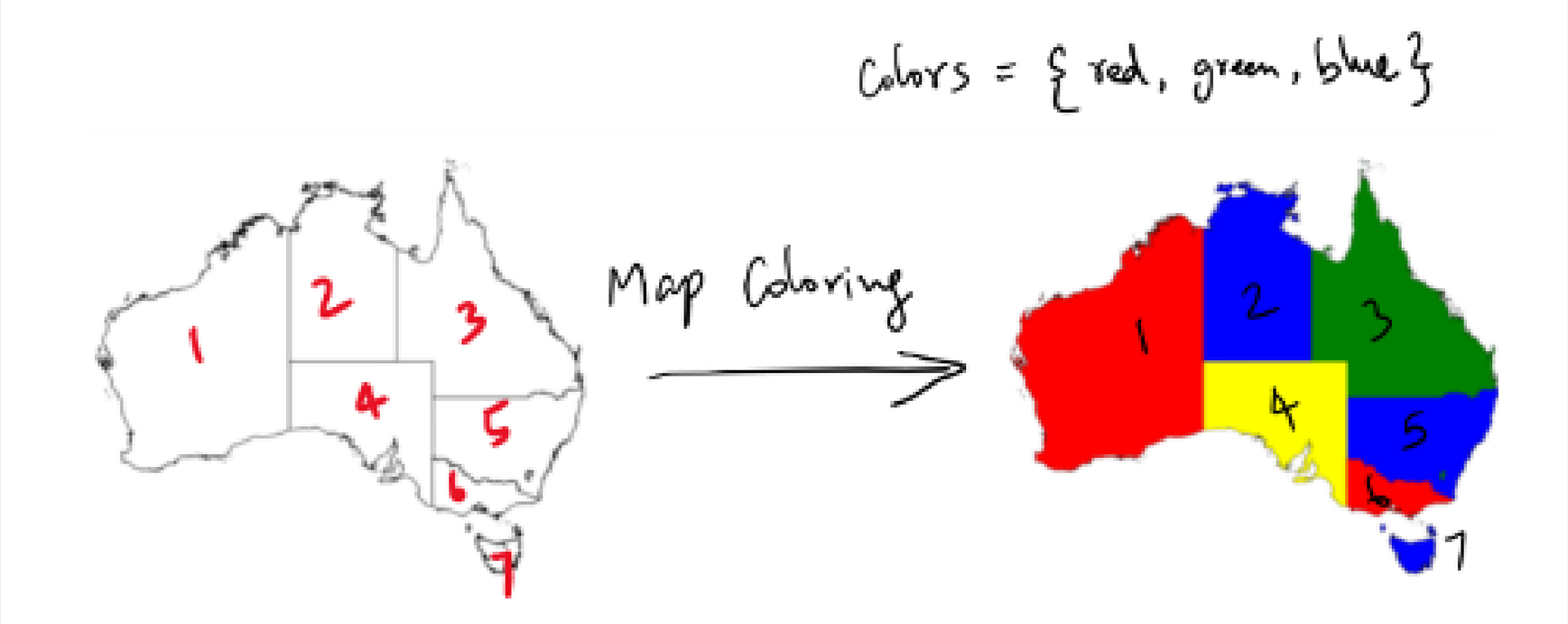
Let’s visualize the process together!



https://n-queen-five.vercel.app/visualize

TUTORIAL QUESTION 5

Given N-regions and M-colors, we want to fill in following map with colors such that no two adjacent regions have same colors. Formulate this as a search problem and draw the search tree.

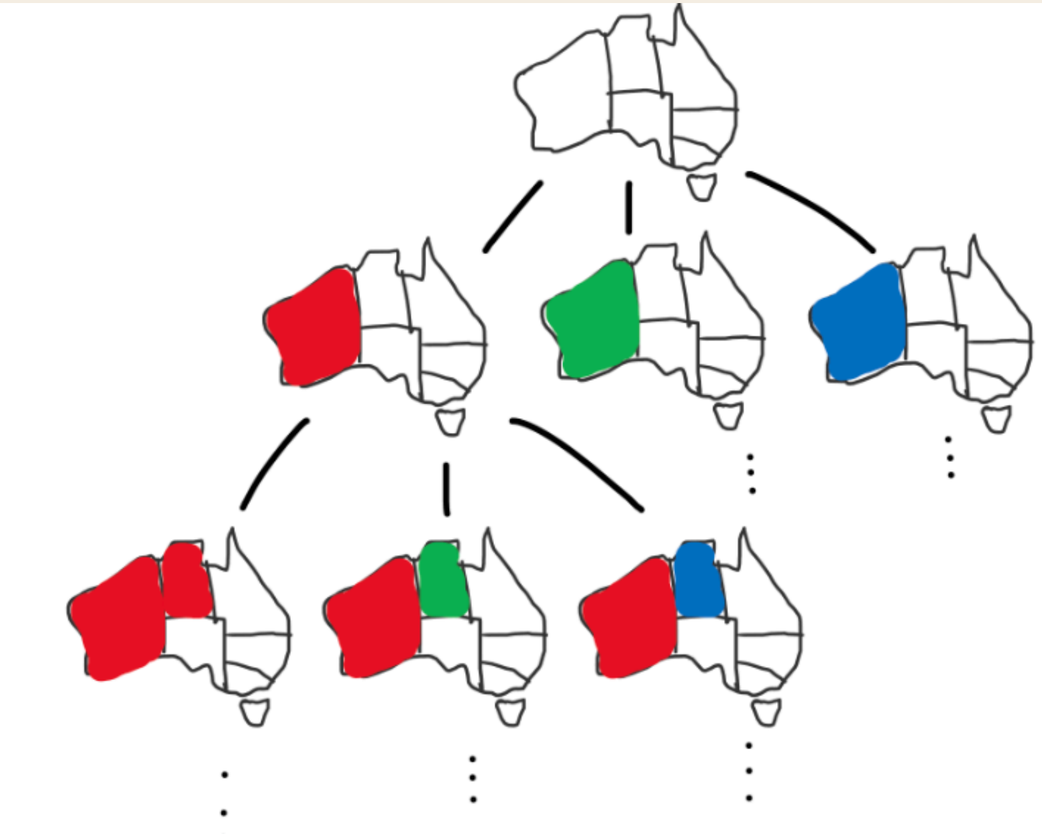


TUTORIAL QUESTION 5

Component	Description
State Space	{All combinations of colour assignments to each region}
Action Space	{Assigning a colour to a region on the map **given the current state of map**}
Successor Function	Return map with painted (selected) region of the choice of colour
Start State	Empty map with no colour on any region
Goal State	All regions are coloured in a way that none of the neighbours have the same colour
Path Cost	0 (Not applicable)

TUTORIAL QUESTION 5

Search tree(Eg. M=3)



- Here we made assumption where each region has numbering, and we will colour the region in the **ascending numbering sequence**.
- We also set constraint to the number of possible colours that we are using.
- In reality, this will be a hard problem with large search tree in terms of branching factor and depth.

QUESTION OF THE WEEK

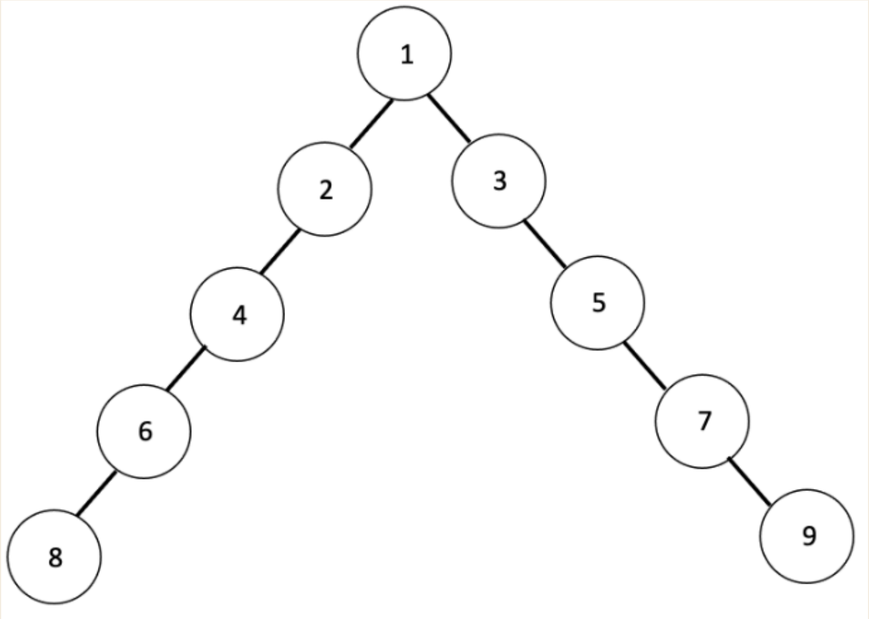
Does BFS always uses more memory than DFS? If not, provide an counterexample.

QUESTION OF THE WEEK

Does BFS always uses more memory than DFS? If not, provide an counterexample.

No.

BFS uses **O(maxWidth)** memory, whereas DFS only uses **O(maxDepth)**. So, when **maxWidth < maxDepth**, BFS uses less memory than DFS, though it is rarely the case.



NEXT WEEK

More Searches – Informed Searches